



VISAYAS
STATE UNIVERSITY

Lesson 1.1

Software and Applications

MODULE 1: APPLICATIONS DEVELOPMENT
Applications Development & Emerging Technologies



Learning Outcome



VISAYAS
STATE UNIVERSITY

1. Identify the fundamental concepts about software.



Software



Pressman (2010) – Software Engineering: A Practitioner's Approach, Chapter 1

Pressman defines **software** not merely as a set of computer programs, but as a **comprehensive product** that includes:

- **Programs:** The actual executable code that performs specific tasks.
- **Data structures:** Organize and manage data effectively, ensuring proper functionality and performance of the software.
- **Documentation:** Guides and manuals for users, developers, and maintainers.
- **Configuration files and support materials:** Enable the software to be installed, operated, and maintained in various environments.

Pressman emphasizes that **software is engineered**, not manufactured. This means it is created through careful planning, design, testing, and maintenance, rather than mass-produced like hardware. The software product is **intangible**, and its quality heavily depends on the **engineering process** behind it.



Software



Sommerville (2015) – Software Engineering (10th Ed.), Chapter 1

Sommerville defines software as a combination of:

- **Computer programs:** The instructions executed by a machine.
- **Associated documentation:** Manuals and reference materials that support both users and developers.

He also makes a distinction between:

- **Bespoke (custom-made) software:** Developed specifically for a particular customer's requirements.
- **Generic software:** Packaged software developed for the open market and sold to a broad range of users.

Sommerville's view highlights the **dual nature of software** as both a **technical artifact** and a **deliverable product**, serving different types of users and contexts.



Software



SWEBOK (2014) – Chapter 1: Software Requirements

According to the **Software Engineering Body of Knowledge (SWEBOK)**, software is:

- A **key component** of a larger **software product**, which is the **result of software engineering processes**.
- This includes not just the executable code, but also:
 - **Requirements specifications**
 - **Architectural and design models**
 - **Source code and binaries**
 - **Documentation**
 - **Configuration and installation procedures**
 - **User manuals and training materials**

SWEBOK treats software as part of a **larger system context**, where it interacts with hardware, users, and other systems. It emphasizes the **importance of process discipline**, quality standards, and professional practices in defining and producing software.



Types of Software



Sommerville (2015) – Software Engineering (10th Ed.), Chapter 1

Sommerville classifies software into **three broad categories**, based on where and how it's used:

1. System Software

Definition: Software that directly manages and controls the hardware components of a computer system.

Examples:

Operating Systems (OS): Windows, macOS, Linux – handle system resources and provide a platform for running application software.

Device Drivers: Allow the OS to communicate with hardware like printers, cameras, and keyboards.

Purpose: Acts as an intermediary between hardware and users or application programs.

Key Feature: Must be reliable and efficient, as all other software depends on it.



Types of Software



2. Application Software

Definition: Software designed to help users perform specific tasks or applications.

Examples:

Productivity Tools: Microsoft Word, Excel, Google Docs.

Media Players: VLC, Spotify.

Games, Educational Apps, and Mobile Applications.

Purpose: Directly serves end users in achieving tasks.

Key Feature: Focus on usability and user interface (UI) design.



Types of Software



3. Embedded Software

Definition: Software that resides within hardware systems (often with no direct user interface) and controls device functions.

Examples:

Automobiles: Engine control units, airbag systems.

Home Appliances: Microwaves, washing machines.

Medical Devices: Pacemakers, infusion pumps.

Purpose: Performs dedicated functions in real-time environments.

Key Feature: Often real-time, resource-constrained, and safety-critical.



Types of Software



Pressman (2010) – Software Engineering: A Practitioner's Approach, Chapter 1

Pressman expands the classification with **more specialized types** of software based on application domains:

4. Real-Time Software

Definition: Software that reacts to input within a strict time limit.

Examples:

Air traffic control systems, anti-lock braking systems, industrial automation.

Purpose: Must respond predictably and quickly to events.

Key Feature: Timing constraints are crucial—failure to respond on time may cause system failure.



Types of Software



5. Business Software

Definition: Software that automates business operations and decision-making.

Examples:

Enterprise Resource Planning (ERP), payroll systems, accounting software.

Purpose: Supports commercial and financial activities.

Key Feature: Handles large volumes of data and focuses on accuracy and security.



Types of Software



6. Engineering/Scientific Software

Definition: Software used for simulations, modeling, and complex calculations in technical fields.

Examples:

MATLAB, CAD (Computer-Aided Design), simulation tools.

Purpose: Enables engineers and scientists to analyze physical systems or perform research.

Key Feature: High accuracy and computation power are essential.



Types of Software



7. Web-Based Software

Definition: Software that runs on web browsers and often relies on internet connectivity.

Examples:

Gmail, Google Docs, Facebook, eCommerce sites.

Purpose: Delivers applications to users over the internet.

Key Feature: Cross-platform compatibility and constant availability.



Types of Software



8. Artificial Intelligence (AI) Software

Definition: Software that simulates aspects of human intelligence.

Examples:

Chatbots, recommendation engines (Netflix, Amazon), autonomous systems.

Purpose: Automate decision-making, learning, and interaction.

Key Feature: Involves machine learning, natural language processing, and data analysis.



Software Characteristics



VISAYAS
STATE UNIVERSITY

Pressman (2010) – Software Engineering: A Practitioner’s Approach, Chapter 1

Pressman outlines the **unique nature of software** compared to other forms of engineered products:

1. Intangible

Unlike physical products such as hardware, software has no physical form.

You **cannot touch or see** the actual software — only its representation (like an app interface or printed code).

This **intangible nature** makes software more difficult to manage, measure, and assess than traditional engineering artifacts.

Example: You can physically inspect a bridge for wear and tear, but inspecting software requires **code review, testing, or analysis tools**.



Software Characteristics



2. Engineered, Not Manufactured

Software is created through **design and logical construction**, not assembled on a production line.

Once software is built, making copies is virtually **cost-free and instantaneous** (e.g., downloading an app).

The “**manufacturing**” **process** of software is essentially the **design and coding stage** — not mass production.

Implication: Traditional manufacturing metrics (e.g., per-unit cost) don't apply to software. The main cost lies in **development, not reproduction**.



Software Characteristics



3. Does Not Wear Out

Software doesn't degrade physically over time like hardware (e.g., rusting metal or worn-out parts).

However, **software can still "fail"** due to:

- Bugs or logic errors

- Compatibility issues with newer systems

- Poor maintenance or updates

Failures often arise because of **environmental changes** rather than internal breakdown.

Analogy: A car engine wears out from use; a software system may "fail" only if changes are made or the environment shifts (e.g., OS update, new user requirements).



Software Characteristics



VISAYAS
STATE UNIVERSITY

4. Evolves Over Time

Software must adapt to **changing user needs, technologies, and business environments**.

This leads to frequent updates, patches, and revisions.

Over time, repeated modifications can make software **complex and harder to maintain** — known as **software entropy**.

Consequence: Continuous evolution requires **structured maintenance processes** and **good documentation** to prevent degradation of quality.



Software Characteristics



Sommerville (2015) – Software Engineering (10th Ed.), Chapter 1

Sommerville complements Pressman's view by focusing on **practical implications** of software's nature:

5. Complex and Flexible

Software systems are **highly complex** due to the large number of interacting components (functions, modules, data structures).

Unlike physical systems, software can be **easily changed**, which makes it **flexible** — but this is a double-edged sword.

Frequent changes often introduce **new bugs**, increase **technical debt**, and make maintenance more difficult.

Insight: Software's flexibility makes it powerful, but also **unstable** if not properly managed with techniques like modular design and version control.



Software Characteristics



VISAYAS
STATE UNIVERSITY

6. High Development Cost, Low Distribution Cost

Developing software is expensive:

- Requires highly skilled personnel

- Involves significant time for design, testing, and debugging

- Needs careful planning and quality assurance

However, once developed, **distributing software is cheap and fast**, especially via the internet.

Example: Creating a new video game may cost millions, but distributing it digitally to millions of users costs almost nothing.



Software Engineering vs Software Development



SWEBOK (2014) – Introduction

Software Engineering is defined as:

“A systematic, disciplined, and quantifiable approach to the development, operation, and maintenance of software; that is, the application of engineering to software.”

This definition highlights that software engineering is **not just about writing code**, but about applying **engineering principles** to ensure that software is:

- **Well-structured**
- **Reliable**
- **Maintainable**
- **Efficient**
- **Scalable**



Software Engineering vs Software Development



SWEBOK emphasizes that **software engineering encompasses** a wide array of disciplines and responsibilities, including:

1. Software Development

- Writing and debugging code
- Implementing algorithms and functionalities

2. Project Management

- Estimating time and cost
- Managing teams and resources
- Tracking progress and risks

3. Quality Assurance

- Testing software
- Ensuring that the product meets user requirements
- Verifying performance and reliability



Software Engineering vs Software Development



VISAYAS
STATE UNIVERSITY

4. Software Process Models

Defining how software is developed and maintained
Choosing between models like Agile, Waterfall, Spiral, etc.

5. Use of Tools

Version control (e.g., Git)
Integrated development environments (IDEs)
Automated testing frameworks

Key Idea: Software engineering is about **building the right software, in the right way, using the right processes and tools.**



Software Engineering vs Software Development



VISAYAS
STATE UNIVERSITY

Sommerville (2015) – Chapter 1

Sommerville draws a **clear distinction** between:

Software Development:

Focuses mainly on **coding and building programs**.

Often practiced by individual developers or small teams.

Emphasizes getting a program to work—**“Does it run?”**

Example: A hobbyist developer building a personal app may only engage in software development—writing code and debugging it.



Software Engineering vs Software Development



VISAYAS
STATE UNIVERSITY

Software Engineering, in contrast:

Is a **structured discipline** that uses:

Methods: Formal approaches to analysis, design, implementation (e.g., Object-Oriented Analysis and Design)

Tools: IDEs, version control, automated build systems, testing tools

Processes: Defined steps for managing development (e.g., Agile, Scrum, DevOps)

Aims to **build large-scale, reliable, and maintainable software systems.**

Involves **team collaboration, documentation, long-term maintenance,** and consideration of real-world constraints.

Example: Developing a banking application that must serve millions of users, ensure data security, and meet government compliance—this requires software engineering.



Software Engineering vs Software Development



VISAYAS
STATE UNIVERSITY

Key Difference:

Software development = "just the coding part"

Software engineering = the entire discipline of building, managing, and evolving software systems



Software Life Cycle



The **Software Life Cycle** (or **Software Development Life Cycle – SDLC**) refers to the **structured sequence of stages** that software goes through — from its initial conception to final deployment and long-term maintenance.

It provides a **framework** for planning, creating, testing, and maintaining software systems.

Dennis (2012) – Systems Analysis and Design, Chapter 1

Dennis describes the **SDLC** as a **standard process** composed of five key phases. This model is commonly used in systems analysis to ensure that software solutions are developed **systematically and efficiently**.



Software Life Cycle



1. Planning

Determine if a new system is needed and feasible.

Identify the scope, objectives, resources, budget, and timeline.

Involves **project initiation** and **team setup**.

Goal: Understand what the organization needs and define the project's direction.

2. Analysis

Gather detailed **requirements** from stakeholders (e.g., users, customers).

Identify problems in the current system and define what the new system must do.

Result: **Functional and non-functional requirements**.

Goal: Understand *what* the system must do.



Software Life Cycle



3. Design

Specify **how** the system will fulfill the requirements.

Includes UI design, database structure, architecture, and business rules.

Produces technical documentation and models.

Goal: Create a blueprint for development.

4. Implementation

The system is **coded, tested, and deployed**.

Developers write code based on the design and integrate system components.

Unit and system tests ensure the system works as intended.

Goal: Turn the design into a working software product.



Software Life Cycle



VISAYAS
STATE UNIVERSITY

5. Maintenance

After deployment, the system must be **maintained and updated**.

Includes bug fixing, performance optimization, and adding new features.

Longest and costliest phase in the life cycle.

Goal: Keep the software reliable, secure, and useful over time.



Software Life Cycle



Pressman (2010) – Software Engineering: A Practitioner's Approach, Chapter 2

Pressman elaborates on **several process models** that represent different ways to **execute the software life cycle**. Each model provides a different structure based on the **nature, risk, and size of the project**.

1. Waterfall Model

Linear and sequential approach: each phase must be completed before moving to the next.

Phases: Requirements → Design → Implementation → Testing → Deployment → Maintenance

Best for: Projects with well-defined, stable requirements.

Limitation: Inflexible to changes once development begins.



Software Life Cycle



2. Incremental Model

Software is developed in **small, manageable parts** (increments).

Each increment adds functionality and is tested and integrated.

Earlier increments can be delivered to users while later ones are still in development.

Best for: Projects where early delivery and feedback are important.

3. Spiral Model

Combines **iterative development** with **risk assessment**.

Each loop in the spiral represents a phase: Planning → Risk Analysis → Engineering → Evaluation.

Suitable for **large, high-risk projects**.

Best for: Projects with high uncertainty and evolving requirements.



Software Life Cycle



VISAYAS
STATE UNIVERSITY

4. Agile Methods

Emphasizes **collaboration, flexibility, iterative releases**, and **customer feedback**.

Agile teams work in **short cycles (sprints)** and continuously adapt.

Examples: **Scrum, Extreme Programming (XP)**

Best for: Projects that require frequent updates, customer input, and fast delivery.



Software Life Cycle



SWEBOK (2014) – Chapter 10: Software Engineering Models and Methods

SWEBOK describes **software life cycle models** as **abstractions of the software development process**. These models help teams visualize and plan the progression of a project from start to finish.

Each model:

Shows **how activities** like requirements gathering, design, coding, testing, and maintenance are organized.

Defines **entry and exit criteria** for each phase.

May support **linear, iterative, or evolutionary** workflows depending on project needs.



Software Life Cycle



VISAYAS
STATE UNIVERSITY

SWEBOK emphasizes that there is **no one-size-fits-all model**. The right model depends on:

- Project size and complexity

- Risk levels

- Client involvement

- Technology used

- Team size and experience



Software Quality Attributes



VISAYAS
STATE UNIVERSITY

Software quality attributes are the **non-functional requirements** (NFRs) that describe **how** the software performs its tasks rather than **what** it does. While functionality is essential, high-quality software must also be **reliable, efficient, usable, and maintainable**.

SWEBOK (2014) – Chapter 11: Software Quality

SWEBOK outlines **six key quality attributes**, also known as the **ISO/IEC 9126 quality model**.

1. Functionality

The software's ability to **perform the tasks it was designed for**.

It includes:

Correctness: Does it do what users need?

Interoperability: Can it work with other systems?

Compliance: Does it meet legal or regulatory standards?

Example: A payroll system must correctly calculate employee salaries and taxes — anything less compromises functionality.



Software Quality Attributes



2. Reliability

The ability of the software to **perform consistently and correctly** under expected conditions.

Includes:

Fault tolerance: Can it recover from errors?

Availability: Is it up and running when needed?

Example: A hospital system must not crash in the middle of a patient record update.



Software Quality Attributes



3. Usability

Measures how **easy and satisfying** the software is to use.

Includes:

- Learnability

- Accessibility

- Aesthetics

- Consistency

Example: A user-friendly e-learning platform allows teachers to upload materials easily without needing technical support.



Software Quality Attributes



4. Efficiency

Refers to the **performance of software in terms of resource usage** (e.g., CPU, memory, bandwidth).

Includes:

- Response time

- Throughput

- Resource optimization

Example: A website that takes 10 seconds to load is inefficient and may lose users.



Software Quality Attributes



5. Maintainability

How easy it is to **modify, update, fix, or enhance** the software.

Includes:

- Code readability and structure

- Modular design

- Documentation quality

Example: Software that's poorly documented and hard to understand is more expensive and time-consuming to maintain.



Software Quality Attributes



6. Portability

The ease with which software can be **transferred from one environment to another** (OS, hardware, platform).

Includes:

- Installability

- Adaptability

Example: An app that runs on both Android and iOS without major changes is portable.



Software Quality Attributes



6. Portability

The ease with which software can be **transferred from one environment to another** (OS, hardware, platform).

Includes:

- Installability

- Adaptability

Example: An app that runs on both Android and iOS without major changes is portable.



Software Quality Attributes



Kleppmann (2021) – Chapter 1: Designing Data-Intensive Applications

Kleppmann focuses on modern software systems that deal with **large-scale data** and emphasizes the following:

1. Reliability

Software must behave correctly, even when:

- Hardware fails

- Users make mistakes

- Network is unstable

In data-intensive applications, reliability also includes:

- Data integrity**

- Fault tolerance**

- Consistent backups and replication**

Example: A banking application must never lose or corrupt transaction data, even during system crashes.



Software Quality Attributes



2. Scalability

The system's ability to **handle increasing loads**, users, or data volume.

Types of scalability:

Vertical: Adding more resources (CPU, RAM)

Horizontal: Adding more machines or servers

Example: A social media app must scale to support millions of concurrent users during peak times.

3. Maintainability

Kleppmann reiterates that software must be:

Easy to fix

Easy to extend

Easy to understand

Example: A team should be able to quickly patch a security issue or add a new feature without breaking existing ones.



Software Quality Attributes



Sommerville (2015) – Chapter 1: Software Engineering

Sommerville introduces **dependability-related** and **user-focused** quality attributes:

1. Dependability

Includes:

Reliability

Availability

Security

Safety

Critical in **safety-critical and mission-critical** systems (e.g., aviation, medicine).

Example: Aircraft control systems must be dependable — failure could lead to loss of life.

2. Efficiency

Similar to SWEBOK's definition: fast response time, minimal resource use.



Software Quality Attributes



3. Acceptability

A more **user-oriented** concept.

Users will only adopt software if it:

- Meets their needs

- Is affordable

- Runs on their devices

- Has acceptable performance and interface

Example: Software may be technically perfect, but if users find it hard to use, it will fail in the market.



Software Quality Attributes



VISAYAS
STATE UNIVERSITY

4. Security

The ability to **protect data and operations** against unauthorized access, attacks, or misuse.

Includes:

- Authentication

- Authorization

- Encryption

- Secure coding practices

Example: An e-commerce platform must protect customer data and prevent fraud.



Questions?

